

PERTEMUAN 3

Multilayer Perceptron dengan Tensorflow

TUJUAN PRAKTIKUM

Implementasi Studi Kasus MLP dengan Tensorflow & Keras

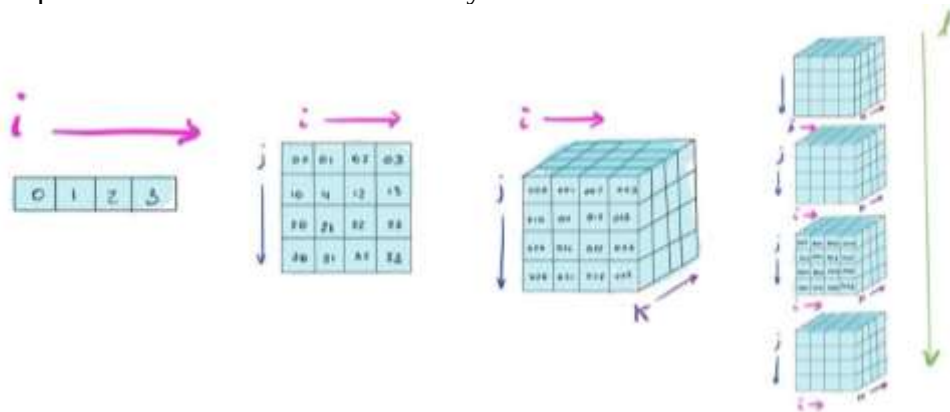
TEORI PENUNJANG DAN MATERI PRAKTIKUM

Pustaka Tensorflow dan Keras untuk Machine Learning

Tensor

Tensors adalah bentuk **struktur data dasar** di dalam TensorFlow, dimana Semua data dienkapsulasi dalam tensor. Pada TensorFlow, tensors diklasifikasikan ke dalam dua bentuk yaitu *constant tensors* dan *variable tensors*:

- *Constant tensor* didefinisikan memiliki value dan dimensi yang tidak dapat diubah(*unchangeable*), dan *variable tensor* didefinisikan memiliki value yang dapat diubah/modifikasi dan dimensi yang tidak dapat diubah(*unchangeable*)
- Dalam Neural network, *variable tensors* secara umum digunakan sebagai matriks untuk menyimpan bobot parameter dan informasi lainnya, dan merupakan tipe data yang dapat di latih sedangkan *constant tensors* dapat digunakan sebagai variabel yang menyimpan nilai-nilai parameter atau struktur data lainnya.



Importing TensorFlow

```
import tensorflow as tf
tf.config.list_physical_devices('CPU')
```

Creating Constant Tensor

Metode umum untuk membuat sebuah constant tensor adalah sebagai berikut:

```
tf.constant()
```

tf.constant(value, dtype=None, shape=None, name='Const', verify_shape=False):

- value: value
- dtype: data type
- shape: tensor type
- name: constant name
- verify_shape: Boolean value, used to verify the shape of a value. The default value is False. If verify_shape is set to True, the system checks whether the shape of a value is consistent with the shape. If they are inconsistent, the system reports an error.

```
# Create a 2x2 matrix with values 1, 2, 3, and 4.
const_a = tf.constant([1, 2, 3, 4], shape=[2,2], dtype=tf.float32)
const_a
# View common attributes.
print("value of the constant const_a:", const_a.numpy())
print("data type of the constant const_a:", const_a.dtype)
print("shape of the constant const_a:", const_a.shape)
print("name of the device to generate the constant const_a:", const_a.device)
```

Membuat sebuah object list menggunakan variable list atau numpy, kemudian mengkonversikan object tersebut ke dalam sebuah tensor. Method berikut dapat mengkonversi sebuah nilai yang diberikan ke dalam sebuah tensor menggunakan `tf.convert_to_tensor` dan dapat pula digunakan untuk mengkonversi sebuah data dengan tipe data di python ke dalam tipe data yang tersedia di dalam TensorFlow.

```
tf.convert_to_tensor(value, dtype=None, dtype_hint=None, name=None)
```

- `value`: value to be converted.
- `dtype`: data type of the tensor.
- `dtype_hint`: optional element type for the returned tensor

```
# Create a list.
list_f = [1,2,3,4,5,6]
list_f
#View the data type.
type(list_f)
tensor_f = tf.convert_to_tensor(list_f, dtype=tf.int32)
tensor_f
list_g = [[1,2],
          [3,4],
          [5,6]]
list_g
tensor_g = tf.convert_to_tensor(list_g, dtype=tf.float32)
tensor_g
```

Creating a Variable Tensor

Dalam TensorFlow, variabel dioperasikan menggunakan method/class **tf.Variable**. `tf.Variable` merupakan sebuah fungsi yang dapat menghasilkan sebuah variable pada tensor dimana untuk nilai dari variabel ini mutable atau dapat diubah/modifikasi dengan menjalankan operasi-operasi aritmatikan pada `tf.Variable`. Nilai variabel dapat dibaca dan di ubah.

```
# Create a variable. Only the initial value needs to be provided.
var_1 = tf.Variable(const_a)
var_1 # var_1.read_value()
var_2 = tf.Variable(list_g)
var_2
# Read the variable value.
print("Value of the variable var_1:\n",var_1.read_value())
#Assign a variable value.
#var_value_1=[[val1, val2],[val3, val4]] #must be the same shape
#var_1.assign(var_value_1)
print("Value of the variable var_1 after the assignment:\n",var_1.numpy())
#Variable addition
var_1.assign_add(const_a)
var_1.assign_add(tf.ones([2,2]))
var_1
```

Tensor Slicing and Indexing

Slicing

Method untuk **slicing** Tensor diantaranya dapat dituliskan dengan cara:

[start: End] → Ekstraksi data dari posisi awal from (start) Sampai dengan posisi Akhir (end).

[start:end:step] or [::step] → Ekstraksi **slice data** pada interval/step dari awal-akhir.

[::-1]: slices data dari elemen yang paling akhir.

'...': menandakan sebuah data slice untuk Panjang berapapun

```
# Create a 4-dimensional tensor. The tensor contains four images. The size
of each image is 3 x 10 x 10.
tensor_h = tf.random.normal([4,3,10,10])
tensor_h
#Extract the first image.
tensor_h[0,:,:,:]
#Extract one slice at an interval of two images.
tensor_h[::2,...]
#Slice data from the last element.
tensor_h[::-1]
```

Indexing

Berikut ini adalah format dasar dalam pengindeksan pada variable tensorflow:

a[d1][d2][d3]

```
#Obtain the pixel, position in row 2, column 3 in the second channel of the
first image.

#?
```

Jika pada indeks data yang akan diekstrak tidak konsekutif, maka method yang dapat digunakan adalah, `tf.gather` merupakan ekstraksi data yang cukup umum di TensorFlow.

Untuk mengekstraksi data pada dimensi tertentu:

```
tf.gather(params, indices,axis=None)
```

params: input tensor.

indices: index of the data to be extracted.

axis: dimension of the data to be extracted.

```
# Extract the 1st, 2nd, and 4th images([4,3,10,10])
indices = [0,1,3]
tf.gather(tensor_h,axis=0,indices=indices,batch_dims=1)
```

Untuk lainnya, dapat dilihat di tautan berikut:

https://drive.google.com/file/d/10BpMFpxylWn3tki-f6pNxQqi5_8K9Nxy/view?usp=sharing

TensorFlow 2.x . Module Examples for Neural Network

Pada sesi berikutnya kita akan fokus kepada modul **Keras** `tf.keras` untuk melihat fundamental modeling pembelajaran jaringan baik secara konvensional ataupun secara lanjut/advance (Neural network modeling :conventional & deeplearning).

a. Building a Model Neural Network Model by Stacking

Cara yang paling umum untuk membangun model NN adalah dengan menggunakan metode stack layers menggunakan: **tf.keras.functional**. Model *machine learning* dibangun secara umum menggunakan **tf.keras.Input** dan **tf.keras.Model**, dimana lebih kompleks bila dibandingkan dengan **tf.keras.Sequential** namun memiliki efek yang baik pada performa pelatihan Variabel dapat menjadi input pada saat yang bersamaan atau di fase berbeda, dan data dapat menjadi output. Functional models lebih menjadi referensi jika kita memiliki lebih dari 1 output model yang dibutuhkan.

Model stacking (.Sequential) vs Functional model (Model)

tf.keras.Sequential model Adalah simple **stack of layers** dan tidak dapat merepresentasikan model lainnya. Kita dapat membangun topologi model kompleks dengan modul **Keras functional model** seperti:

- Multi-input models.
- Multi-output models.
- Models with shared layers.
- Models with non-sequential data flows (for example, residual connections).

Pada contoh berikut kita akan menggunakan model simple stack:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(4,)))
model.add(layers.Dense(2, activation='relu'))
model.add(layers.Dense(3, activation='relu'))
model.add(layers.Dense(4, activation='softmax'))

model.summary()
```

Equivalent to this process :

```
#Use the output of the previous layer as the input of the next layer.
x = tf.keras.Input(shape=(4,))
h1 = layers.Dense(2, activation='relu')(x)
h2 = layers.Dense(3, activation='relu')(h1)
y = layers.Dense(4, activation='softmax')(h2)
model2 = tf.keras.models.Model(x, y)
model2.summary()
```

b. Building a Network Layer (tf.keras.layers)

Modul **tf.keras.layers** digunakan untuk **mengkonfigurasi layer NN**. Berapa class/method yang umum diantaranya:

- **tf.keras.layers.Dense**: builds a fully connected layer.
- **tf.keras.layers.Conv2D**: builds a two-dimensional convolutional layer.
- **tf.keras.layers.MaxPooling2D/AveragePooling2D**: builds a maximum/average pooling layer.
- **tf.keras.layers.RNN**: builds a recurrent neural network layer.
- **tf.keras.layers.LSTM/tf.keras.layers.LSTMCell**: builds an LSTM network layer/LSTM unit.
- **tf.keras.layers.GRU/tf.keras.layers.GRUCell**: builds a GRU unit/GRU network layer.
- **tf.keras.layers.Embedding**: converts a positive integer (subscript) into a vector of a fixed size, for example, `[[4],[20]]->[[0.25,0.1],[0.6,-0.2]]`. The embedding layer can only be used as the first model layer.

- **tf.keras.layers.Dropout**: builds the dropout layer.

Beberapa konfigurasi hyperparameter dalam **tf.keras.layers** diantaranya:

- **activation**: sets the activation function for the layer. By default, the system applies no activation function.
- **kernel_initializer** and **bias_initializer**: initialization schemes that create the layer's weights (kernel and bias). The default initializer is Glorot_uniform.
- **kernel_regularizer** and **bias_regularizer**: regularization schemes that apply to the layer's weights (kernel and bias), such as L1 or L2 regularization. By default, the system applies no regularization function.

f.keras.layers.Dense

Layer Dense mengimplementasikan operasi: **output = activation(dot(input, kernel) + bias)** dimana **activation** adalah sejumlah aplikasi fungsi aktivasi yang diberikan dengan argument fungsi, kernel, matrix bobot dan bias yang dihasilkan dari topologi layer.

Konfigurasi utama untuk parameter dalam **tf.keras.layers.Dense** diantaranya:

- **units**: number of neurons.
- **activation**: sets the activation function.
- **use_bias**: indicates whether to use bias terms. Bias terms are used by default.
- **kernel_initializer**: initialization schemes that create the layer's weight (kernel).
- **bias_initializer**: initialization schemes that create the layer's weight (bias).
- **kernel_regularizer**: regularization schemes that apply to the layer's weight (kernel).
- **bias_regularizer**: regularization schemes that apply to the layer's weight (bias).
- **activity_regularizer**: regular item applied to the output, a Regularizer object.
- **kernel_constraint**: a constraint applied to a weight.
- **bias_constraint**: a constraint applied to a weight.

Berikutnya kita akan coba membuat jaringan terhubung sempurna (**fully connected layer**) yang berisikan **4 neurons** dan **activation function to sigmoid**. Fungsi aktivasi dapat dipanggil dengan memberikan nama fungsi sebagai parameter, sebagai contoh **sigmoid** atau menggunakan object function, **tf.sigmoid**.

```
x = tf.keras.Input(shape=(4,))
hidden1 = layers.Dense(4, activation='sigmoid')(x)
hidden2 = layers.Dense(4, activation=tf.sigmoid)(hidden1)
y = layers.Dense(1, activation='sigmoid')(hidden2)
model3 = tf.keras.models.Model(x, y)
model3.summary()
```

c. Network Layer Implementation

Berikutnya kita akan mencobakan NN sederhana dengan sebelumnya kita membuat jaringan NN terlebih dahulu. Untuk data yang digunakan adalah data iris. Dapat diakses pada tautan: <https://drive.google.com/drive/folders/1hRZED4AezCR9RjTcy7JFBhJ-X2qqWse2?usp=sharing>

Import module

```
from keras.models import Model
from keras.layers import Input, Dense
import pandas as pd
from tensorflow.keras.utils import to_categorical
import numpy as np
```

Data Sourcing

```
data = pd.read_csv('iris.data').values
```

Data Preparation

```

rand = np.arange(149)
np.random.shuffle(rand)
c = 0
copy = data.copy()
for i in rand:
    data[c] = copy[i]
    c = c+1
print(data)
X = data[:, :4]
y = data[:, 4]
#print(y)
labels = {}
cnt = 0

for i in y:
    if i not in labels:
        labels[i] = cnt
        cnt = cnt + 1

#print("="*50)
#print(labels)
for i in range(y.shape[0]):
    y[i] = labels[y[i]]
print(y)
y = to_categorical(y)
#print(y)
X= np.array(X, dtype="float64")
y= np.array(y, dtype="float64")
print(X.shape, y.shape

```

Model Building

```

inp = Input(shape=(4))
x = Dense(32, activation="relu")(inp)
op = Dense(3, activation="softmax")(x)
model= Model(inputs=inp, outputs=op)
model.compile(optimizer="rmsprop", loss="categorical_crossentropy",
metrics=['acc'])
model.summary()

```

Model Training

```
model.fit(X, y, epochs=100)
```

Prediction

```

pred = model.predict(np.array([1.4,2,0.5,1]).reshape(1,-1))
print(pred)
print(np.argmax(pred))
pred[0][np.argmax(pred)]

```

Model Saving

```
model.save('model.h5')
```

Studi Kasus Backpropagation untuk Dataset MNIST menggunakan TensorFlow

Berikutnya kita akan mempersiapkan data yang akan diimplementasikan dengan menggunakan Algoritme MLP-Backpropagation menggunakan python dan Pustaka tensorflow/keras. Model MLP-BP akan dilatih menggunakan data MNIST yang merupakan citra angka dalam bentuk digital yang dapat diakses secara bebas (disediakan saat praktikum dan simpan data tersebut pada direktori anda). Berikut Langkah-langkah yang perlu dilakukan:

- Membaca data dalam format udf terkompresi (*.gz) kemudian dikonversi ke dalam bentuk array (dengan pustaka numpy). Sebelumnya panggil Pustaka yang diperlukan seperti numpy, keras, matplotlib dll

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
import os
import gzip
cwd = os.getcwd()
print(cwd)
```

- Total dataset yang dapat digunakan untuk pelatihan adalah sebanyak 60000 citra, dengan masing-masing citra berukuran 28 piksel x 28 piksel. Komponen data set terdiri dari dataset citra sebagai data fitur, dan juga dataset label yang menunjukkan target output/label ataupun kelas dari citra. Karena jumlah digit decimal berjumlah 10, maka ada sebanyak 10 kelas sebagai representasi dari citra digit angka. Baris kode berikut ini adalah untuk membaca dan mengkonversi data ke dalam bentuk array baik untuk label-set maupun image-set.

```
# Baca label
labels_path = 'train-labels-idx1-ubyte.gz'
f=gzip.open(labels_path,'rb')
file_content=f.read()
labels = np.frombuffer(file_content, dtype=np.uint8, offset=8)
length = len(labels)
print(labels.shape)
```

```
# Baca Fitur Data:
images_path = 'train-images-idx3-ubyte.gz'
f=gzip.open(images_path,'rb')
image_content=f.read()

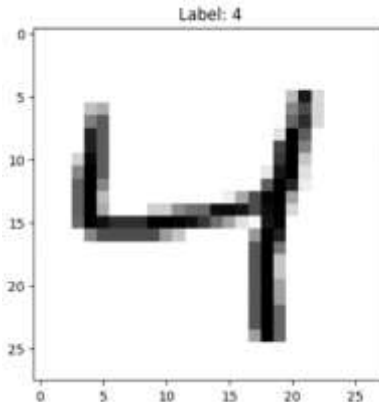
img = np.frombuffer(image_content, dtype=np.uint8,
                    offset=16).reshape(length, 28, 28)

features = np.frombuffer(image_content, dtype=np.uint8,
                        offset=16).reshape(length, 784)

print(features.shape)
```

- Selanjutnya code berikut adalah untuk memperlihatkan plot salah satu citra beserta dengan labelnya. variabel "pos" menunjukkan indeks data yang akan diplot

```
# Plot data citra pada dataset dengan indeks ke-2
pos = 2
image = img[pos].squeeze()
plt.title('Label: %d' % (labels[pos]))
plt.imshow(image, cmap=plt.cm.gray_r)
```



Gambar 1 Hasil plotting dataset MNIST

Tugas Mandiri:

Terapkan Kode di atas dan hasil file *.ipynb dan selanjutnya mengerjakan Kuis (di class atau di gform (jika masih terkendala akses masuk ke course CI di class)).

1. Pengerjaan Kuis di class:
<https://class.ipb.ac.id/mod/quiz/view.php?id=100936>
2. Pengerjaan Kuis di Gform:
<https://forms.gle/o6SH4o3Gs3araseU8>

Batas waktu pengumpulan 4 September 23.59 wib

Tugas kelompok:

3. Bentuk kelompok (2-3) orang satu kelompok
4. Terapkan Pelatihan model MLP menggunakan Tensorflow untuk Data: (**Pilih salah satu saja**):
 - a. Data tanda tangan masing-masing anggota kelompok (menjadi 2-3 kelas)
 - b. Data Wajah anggota masing-masing anggota kelompok (menjadi 2-3 kelas)
5. Tentukan arsitektur yang digunakan untuk melatih model pembelajaran MLP untuk kedua dataset di atas, kemudian jelaskan untuk Kode yang diberikan (dapat dijelaskan dalam bentuk snippetcode).
6. File yang dikumpulkan:
 - Kode program untuk penerapan MLP data yang dipilih
 - Penjelasan presentasi hasil dalam bentuk video (link tautan video yang disimpan di drive)
7. Tugas dikumpulkan pada Pekan ke 5 (18 September 2026) pada gform: <https://forms.gle/M38raF7oUsGjFry26>
8. Yang mengumpulkan cukup salah satu anggota kelompok dengan menuliskan semua NIM anggota